University of Guelph

Computer Science Department

CIS*6530 - Threat Intel & Risk Analysis

Machine Learning Pipeline for Malware Extraction and Classification of Advanced Persistent Threats

Submission 5

(20%)

Team Members
Jacob Drobena | jdrobena@uoguelph.ca
Nafis Ahmed Awsaf | nawsaf@uoguelph.ca

Professor: Dr. Ali Dehghantanha

27th November 2025

Table of Contents:

I. Introduction:

Advanced Persistant Threats (APT's) represent some of the most notable and sophisticated cyber adversaries that have been documented. Catagorized by long-term cyber attack campaigns by coordinated and well skilled malicious actors. APT's often backed by nation state resources to disrupt or perform espionage activity against opposing governments, militaries, and critical infrastructure. Their use of customized malware and advanced tactics require cyber-defence entities to develop intelligence-driven analytics, malware behavior catalogues, and machine-learning approaches in order to better act on emerging or ongoing threats.

This project outlines the works of various stages of APT malware analysis to formulate a comprehensive pipeline for analyzing APT catagorized malware using opcode feature extraction and machine learning techniques to extract malware behaviour to then effectively distinguish their related APT families. Our work focuses on selected Group Set 5 composed of 38 distinct APT groups spanning China, North Korea, the United States, the Middle East amongst others. Through threat inteligence review of the noted APT's, designed automated malware retrieval and extraction tools, data preprocessing and Machine Learning (ML), this project provides an end to end pipeline to evaluated the effectiveness of ML based cyber-threat classification based on Indicator of Compromise (IOC) sharing.

The remainder of the paper is as followed: Section II - Background provides the necessary highlights of previous works relating to this assignment for submissions 1-3. Section III - Data Preperation outlines the data preprocessing steps to properly structure the data,clean and refine the opcode dataset. Section IV - Classic ML Methodology describes a synopsis of previous submissions to apply classification algorithms (SVM, KNN (k=3.5), Decision Tree) on 1-Gram and 2-Gram OpCodes. Section V - Expand on the Deep Learning Methodology proposed in the paper "Deep Android Malware Detection" for opcode based APT classification. Section VI - Results compares the previous classification algorithms with the revised adaptation of deep CNN ML modeling provided in the noted paper.

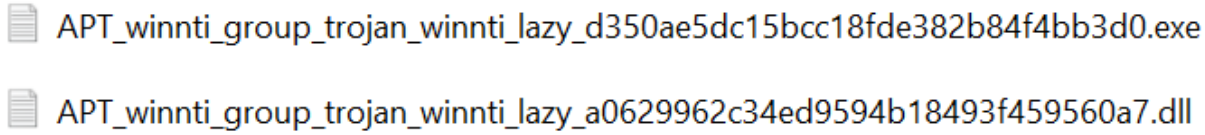
II. Background and Project Overview

2.1 Threat-Intelligence of APT Campaigns

This step involves the research and investigation of our selected group of APT's consisting of 38 APT groups, to which we documented their targets, their techniques, their procedures and other relevant data using MITRE ATT&CK and supporting vendor intelligence. With this we were able to pull understanding and contexts from researchers providing a foundation and approach to better review well documented APT groups like Lazarus Group, Winnit and APT32 and allowed understanding and time to investigate lesser documented groups like Sowbug or Whitefly.

2.2 Automations for APT linked malware retrieval

From a foundation in part 1, we utilized the gathered sources for each APT and collected documented Indicators of Compromise (IOC's) in the form of filehashes. Categorically we listed the hashes associated with each APT and used API queries from VirusTotal, VirusShare, and MalwareBazaar running a custom script to retrieve the zipped malware properly labled using naming conventions"APT_<group name>_"malware common name"_<MD5>.<extension>" before moving the file to their specific folder of 'executable malware' or 'Other payloads'. All samples provided by the virus sharing platforms were delivered in password-protected zip-files with the password for all zip-compressed malware samples being "infected".



2.3 Opcode Extraction

Within an isolated KALI Linux environment, the malware samples were decrypted and processed to extract low-level instruction opcodes. An open-source Ghidra opcode extraction tool was re-engineered to support batch processing to output lightweight .opcode output formats. This iterative retrieval from parts 2.2 and 2.3 were continued in part 4 and 5.

III. Dataset & Preprocessing:

Conducted in part 4, the extracted opcode dataset required extensive preprocessing to process integrity. Re-investigation of the extracted files determined significant crossover of duplicate filehases with IOC indicators. Duplicate hashes were condensed into one sample with copying activity across APTs met with review of the malware and determination if it was deleted as a whole or just linked to a specific APT. Near duplicate samples were identified utilizing Jaccard similarity removing overlapping threat inteligence which propted further investigation that linked groups like Promethium with Neodymium and Whitefly with TG-3390 to limit threat intel feeds with overlapping evedence and campaigns. This allowed for more APT specific malware more unique to the specified APT family. The refined dataset consisted of 777 opcode files across 34 APT groups.

073cbd6533835a3df9a0c569c	ThreatGrou	\APT_Promethium_trojan_graf	within-APT-outlier	7.231994
073cbd6533835a3df9a0c569c	Whitefly	\APT_Promethium_Win32Stro	within-APT-outlier	7.231994
073cbd6533835a3df9a0c569c	Whitefly	\APT_Carbanak_trojan_mint_z	within-APT-outlier	4.185978
073cbd6533835a3df9a0c569c	Whitefly	\APT_LotusBlossom_trojan_m	within-APT-outlier	3.855278
073cbd6533835a3df9a0c569c	Whitefly	\APT_StealthFalcon_trojan_de	within-APT-outlier	3.739271
073cbd6533835a3df9a0c569c	Whitefly	\APT_APT18_9603b62268c2b	within-APT-outlier	3.479871
073cbd6533835a3df9a0c569c	Whitefly	\APT_APT38_trojan_alreay_do	within-APT-outlier	3.448888

Additional steps determined ML training required more logs then specific APT groups were able to train on, hence during ML training we dropped off any underrepresented APT group that had less then 15 samples. This refined dataset was consolidated into 609 malware samples across 11 APT groups.

apt	count			
DarkHotel	186		Thrip	10
winnti	125		C-36	9
Promethiur	121		Neodymiur	8
Taidoor	38		Zirconium	7
Machete	27		PLATINUM	6
APT32	25		Lazarus	6
BlackTech	24		DustStorm	5
APT18	23		Rancor	5
Carbanak	19		Higaisa	5
APT38	16		Kimsuky	4
Poseidon	16		Leviathan	3
Strider	15		TropicTroo	3
StealthFalc	15		Whitefly	2
Equation	14		STOLENPE	2
APT37	13		DragonOK	1
ThreatGrou	11		Sowbug	1
LotusBloss	11		TA551	1

With this restructuring opcode sequences were transformed into n-gram feature vectors generating both unigram (1-gram (MOV, PUSH, CALL)) and bigram (2 gram (PUSH MOV, MOV CALL)) feature representations.

token	count
add add	17942184
mov mov	4713097
int3 int3	1849983
push push	1272044
call mov	1030369

IV. Classic ML Methodologies and Approaches:

This section outlines the methodology used to evaluate opcode based classification using ML models: Decision Tree, k-Nearest Neighbors (KNN), and Support Vector Machine (SVM). All models were trained on the preprocessed opcode dataset formulated in Section III, using both 1-gram and 2-gram opcode representations.

4.1 Feature Extraction

Opcode sequences were converted into numerical feature vectors using term frequency–inverse document frequency (TF-IDF) representations [1]. This vectoization of each step quantifies the relative importance of each opcode or opcode cluster within a malware sample allowing for ML models to capture trends across malware samples in order to associate this activity amongst APT groups.

- 1-gram TF-IDF: individual opcode mnemonics (MOV, PUSH, CALL)
- 2-gram TF-IDF: pairs of consecutive opcodes (PUSH MOV, MOV CALL)

```
    tr_idx, te_idx = train_test_split(
        df_local.index, test_size=TEST_SIZE, random_state=SEED,
    )
    df_tr = df_local.loc[tr_idx].copy()
    df_te = df_local.loc[te_idx].copy()

    print(f"\n[BUILD] {ngram}-gram TF-IDF: train={df_tr.shape[0]
    tfidf = TfidfVectorizer(
        analyzer="word",
        ngram_range=(ngram, ngram),
        token_pattern=TOKEN_PATTERN
```

4.2 Dataset Splitting

Both 1-gram and 2-gram generated dataframes were split using a 80/20 stratified split. The stratification to ensure equal APT distribution across training and testing datasets.

4.3 Machine Learning model

Three models were trained and evaluated used and adapted from in class examples.

Decision Tree Classifier: using root node representation with each additional node representing a decision or test on these attributes with the outcomes.

Due to our unbalanced dataset we tested various classifiers including max depth, class weights and number of leaf nodes. Through structured testing we determined unrestricted depth and unrestricted leafs like the class example with deep subtrees often captured the fine distinctions between APT families. Often yielding the best results of the three.

k-Nearest Neighbors (KNN): stores all training samples and makes predictions based on the majority label among the four closest neighbors under Euclidean distance [3]. KNN rubric (k=3.5) was required to round to a valid integer being 4. Our KNN model followed the in class example closely with added print statements for troubleshooting.

Support Vector Machine (SVM): identifies a set of support vectors that define non-linear decision boundaries in TF-IDF space

V. Deep Learning Methodology

This section details the end-to-end methodology used to implement a convolutional neural network (CNN)–based opcode classifier modeled after the *Deep Android Malware Detection* framework [4]. This section is to build upon previous ML pipeline to integrate this CNN

modeling comparing and contrasting the results with the classifiers.

5.1 Opcode File Ingestion

All .opcode files produced in earlier stages were recursively scanned and loaded outlined in Section II and 4.1. Opcode mnemonics were iterated through per file, metadata collected including filename, APT label, file type and opcode statistics were collected. Opcode sequences were concatenated into single space dataframes viable for ingestion.

5.2 Label Filtering and Stratification

Aligned with submission 4, APT groups fewer then 15 samples were considered unviable and skewed the data, hence removed to maintain a stable training. Remaining APT families were viable and distinct via dataset pruning in Section III.

Following methodology in the example CNN pipeline, a 70/20/10 split of the 609 malware samples across 11 APT groups was conducted.

5.3 Opcode Vocabulary Construction

All opcode sequences were tokenized on whitespace using the pattern \S+, ensuring that each assembly mnemonic (MOV, PUSH, CALL, etc.) had its own unique token identifier. Denoted as DalvikOpcodes.txt was formulated for our extended vocabulary, the original paper highlighted ~180 dalvik opcodes where as our dataset has compiled a list of 528 opcode types.

IE. vocabulary txt

<PAD> = 0

<UNK> = 1

…

This tokenization of mnemonic drastically increases the CNN model to effectively represent more data with lower hardware constraints. To align with the example opcode document was transformed into a fixed-length integer sequence

5.4 CNN Architecture

The CNN architecture followed the design principles of the Deep Android Malware Detection model, adapted for APT-based Windows opcode sequences. An embedding layer transformed integer-encoded opcode sequences into dense vector representations, Embedding dimension emb_dim, dataframe shape - max_len, kernel_size, dropout_rate, class_weight and epochs were all structured to align with android malware detection.

Pipeline consists of data ingestion -> dataframe formation -> training -> testing model -> results output.

5.4 Hyperparameter Testing

Due to our dataset differing significantly from the paper’s dataset, we loop our dataset dataset through hyperparameter sensitivity analysis to produce optimal parameter sets following the methodology of previous hyperparameter testing [5]. This was decided to due to more

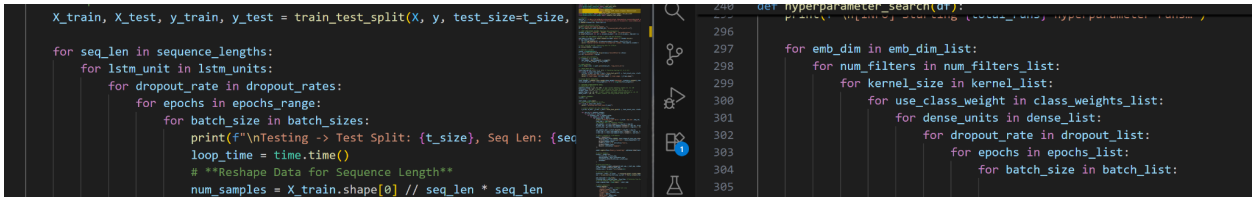
extended and unique testing requirements ie.

- Paper dataset: benign vs malware (2 classes)
- Our dataset: 11 malware APT families (multi-class)
- Paper dataset: ~180 Dalvik opcodes
- Our dataset: 528 opcode types (~3x larger)
- Paper samples are short sequences
- Our sequences are 10x longer on average

To better evaluate optimal results while evaluating a larger array of parameters we iteratively loop through the below.

```
emb_dim_list       = [8, 32, 64]       # 8 is OG paper, class CNN uses deeper feature maps (no embedding)
num_filters_list   = [32, 64, 128]  # class uses 16→32 filters; 64 matches paper
kernel_list        = [3, 8]         # class CNN always used kernel 3, 8 paper, test bigger
dense_list         = [16, 32]       # 16 = paper, 64 LSTM test, 32 to look at
dropout_list       = [0.0, 0.5, 0.8]     # simple cnn had no dropout; paper 0.5, old code I checked 0.8
class_weights_list = [True, False]  # was reccomended to balance to allow for generalization
epochs_list        = [10, 20]           # Class example 3 not worth to test, paper 10, previous testing 20
batch_list         = [16]           # Stable google says not worth it to test for CNN
```

The iterative loops are seen below, ie previous example and the new revision. (left old, right new)



```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=t_size,

for seq_len in sequence_lengths:
    for lstm_unit in lstm_units:
        for dropout_rate in dropout_rates:
            for epochs in epochs_range:
                for batch_size in batch_sizes:
                    print(f"\nTesting -> Test Split: {t_size}, Seq Len: {sec
                    loop_time = time.time()
                    # **Reshape Data for Sequence Length**
                    num_samples = X_train.shape[0] // seq_len * seq_len
```

```
def hyperparameter_search(df):

    for emb_dim in emb_dim_list:
        for num_filters in num_filters_list:
            for kernel_size in kernel_list:
                for use_class_weight in class_weights_list:
                    for dense_units in dense_list:
                        for dropout_rate in dropout_list:
                            for epochs in epochs_list:
                                for batch_size in batch_list:
```

Through review the below can be seen

run_id	emb_dim	num_filters	kernel_size	class_weig	dense_unit	dropout_ra	epochs	batch_size	accuracy	precision	recall	f1_score
416	64	128	8	TRUE	32	0	20	16	0.9032258	0.903584	0.9032258	0.8996757125789384
368	64	64	8	TRUE	32	0	20	16	0.8817204	0.9160383	0.8817204	0.8866743428955411
380	64	64	8	FALSE	32	0	20	16	0.8709677	0.8786800	0.8709677	0.8681903520613198
428	64	128	8	FALSE	32	0	20	16	0.8709677	0.8710872	0.8709677	0.8670682081547955
410	64	128	8	TRUE	16	0	20	16	0.8602150	0.8652810	0.8602150	0.8559164661032233
404	64	128	3	FALSE	32	0	20	16	0.8602150	0.8625509	0.8602150	0.8553787531716224
284	32	128	8	FALSE	32	0	20	16	0.8602150	0.8647527	0.8602150	0.8549994720726427
332	64	32	8	FALSE	32	0	20	16	0.8602150	0.8833828	0.8602150	0.8528955656875566
412	64	128	8	TRUE	16	0.5	20	16	0.8494623	0.8770165	0.8494623	0.8516854703308828
236	32	64	8	FALSE	32	0	20	16	0.8602150	0.8477253	0.8602150	0.8492451395677202
422	64	128	8	FALSE	16	0	20	16	0.8494623	0.849648	0.8494623	0.8464460270911884
362	64	64	8	TRUE	16	0	20	16	0.8494623	0.8495104	0.8494623	0.8446273626482212

With ideal results optimal parameters consist of

emb_dim	num_filters	kernel_size	class_weig	dense_unit	dropout_ra	epochs	batch_size	accuracy	precision	recall	f1_score
64	128	8	TRUE	32	0	20	16	0.903226	0.903584	0.903226	0.899676
64	128	8	TRUE	16	0.5	20	16	0.849462	0.877017	0.849462	0.851685

VI. Results and Evaluation

CNN model results

accuracy	precision	recall	f1_score
0.903226	0.903584	0.903226	0.899676
0.849462	0.877017	0.849462	0.851685

Part 4

```
*** Overall Classifier Comparison ***
           Model Feature  Accuracy  Precision  Recall  F1 Score
0  Decision Tree  1-gram     86.67      88.26   86.67     86.68
1            KNN  1-gram     71.11      70.97   71.11     70.89
2            SVM  1-gram     47.78      29.90   47.78     35.80
3  Decision Tree  2-gram     86.67      86.82   86.67     86.31
4            KNN  2-gram     71.11      70.97   71.11     70.89
5            SVM  2-gram     47.78      29.90   47.78     35.80
```

As can be seen above the CNN trained on integer-encoded opcode sequences significantly outperformed the classical models. Under the optimized hyperparameter configuration (embedding dim = 64, 128 filters, kernel size = 8, dense=32, no dropout, class weights enabled), the CNN achieved:
Accuracy: 90.32%
Precision: 90.36%
Recall: 90.32%
F1-Score: 89.97%
With a number of other results as listed in the hyperparameter testing csv with significant high results where tweaking variables shows no indication of overfitting.

The classical machine-learning baselines—Decision Tree, KNN, and SVM—were evaluated across both 1-gram and 2-gram opcode features. The Decision Tree consistently performed the best among the classical models, achieving:
Accuracy: 86.67%
Precision: 88.26%
Recall: 86.67%
F1-Score: 86.68%
KNN produced moderate performance (~71% accuracy), while SVM struggled across both n-gram feature sets with accuracies below 48%

The CNN's improvement can be attributed to several architectural advantages that reduces the hardware constraints that cant be examined improved using unique token identifiers. The results clearly demonstrate that the adapted model example seen in the deep-learning CNN substantially outperforms the classical ML models used in Submission 4. While the Decision Tree does show promise in achieved strong baseline maybe requiring more tuning, the CNN proves ideal in malware classification with 90% accuracy results.

VII. Citations

[1] GitHub. "tf-idf — GitHub Topics." GitHub, https://github.com/topics/tf-idf
[2] schwartz1375. opcode_n-gram_ml. GitHub repository. https://github.com/schwartz1375/opcode_n-gram_ml
[3] "K-Nearest Neighbors (KNN) Algorithm," GeeksforGeeks, 23 Aug. 2025.
[4] Niall McLaughlin, Jesus Martinez del Rincon, BooJoong Kang, Suleiman Yerima, Paul Miller, Sakir Sezer,Yeganeh Safaei, Erik Trickel, Ziming Zhao, Adam Doupé, and Gail Joon Ahn. "Deep android malware detection." In Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy (CODASPY '17), pages 301–308, New York, NY, USA, 2017. Association for Computing Machinery
[5] J. Drobena, and R. E. De Grande, "Real-Time Evaluation of LSTM-Based IDS Using a Live RF VANET SDR-Enabled Testbed," *Department of Computer Science, Brock University, Canada*.